

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ



МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

*Факультет Информационных технологий
Кафедра Информатики и информационных технологий*

направление подготовки

09.03.02 «Информационные системы и технологии»

ПРАКТИЧЕСКАЯ РАБОТА № 10

Дисциплина: BackEnd-разработка

Тема: Логирование веб-приложение на основе ASP.NET Core

Выполнил: студент группы 241 - 333

Конев Максим Дмитриевич
(Фамилия И.О.)

Дата, подпись _____
(Дата) (Подпись)

Проверил: _____
(Фамилия И.О., степень, звание) (Оценка)

Дата, подпись _____
(Дата) (Подпись)

Замечания: _____

Москва

2026

Цель работы

Ознакомиться с методами логирования и их применением в веб-приложениях на платформе ASP.NET Core.

Используемые технологии и инструментальные средства

В работе применялись:

- ASP.NET Core Web API — разработка HTTP API
- C# — язык программирования
- Serilog — библиотека логирования
- Swagger (OpenAPI) — тестирование API
- DataAnnotations — валидация входных данных
- Visual Studio — среда разработки

Ход выполнения работы

1. Общая концепция логирования в ASP.NET Core

Логирование — это процесс записи информации о работе приложения с целью:

- отслеживания выполнения операций
- диагностики ошибок
- анализа поведения системы
- аудита действий пользователя

В ASP.NET Core используется абстракция `ILogger<T>`. Она позволяет:

- писать логи разных уровней
- не зависеть от конкретной реализации (Serilog, NLog и т.д.)
- легко внедрять логирование через DI

2. Внедрение логгера в контроллер

В контроллере логгер подключается через конструктор:

```
public class StudentsController(ILogger<StudentsController> logger) : ControllerBase
```

Рисунок 1. Внедрение логгера в контроллер

Суть:

- контейнер зависимостей автоматически передаёт реализацию
- логгер привязан к конкретному классу (SourceContext)
- в логах видно, откуда пришло сообщение

3. Подключение Serilog

Стандартный логгер ASP.NET Core расширен с помощью Serilog. Регистрация логгера:

```
builder.Services.AddLogging(l =>
{
    l.ClearProviders(); // удаление стандартных провайдеров
    l.AddSerilog(new LoggerConfiguration()
        .ReadFrom.Configuration(builder.Configuration)
        .CreateLogger());
});
```

Рисунок 2. Регистрация логгера

4. Конфигурация логирования

Настройки вынесены в appsettings.json:

```
"Serilog": {
  "MinimumLevel": {
    "Default": "Debug",
    "Override": {
      "Microsoft": "Debug",
      "Microsoft.AspNetCore": "Information",
      "System": "Debug"
    }
  },
  "WriteTo": [
    {
      "Name": "Console",
      "Args": {
        "outputTemplate": "[{Timestamp:yyyy-MM-dd HH:mm:ss.fff zzz} {Level:u3}] {SourceContext} {Message:l} {Properties:j}{NewLine}{Exception}"
      }
    },
    {
      "Name": "File",
      "Args": {
        "path": "logs/app-.log",
        "rollingInterval": "Day",
        "retainedFileCountLimit": 5,
        "rollOnFileSizeLimit": true,
        "fileSizeLimitBytes": 104857600,
        "outputTemplate": "[{Timestamp:yyyy-MM-dd HH:mm:ss.fff zzz} {Level:u3}] {SourceContext} {Message:l} {Properties:j}{NewLine}{Exception}"
      }
    }
  ]
},
```

Рисунок 3. Конфигурация логгера

Разбор конфигурации

- `MinimumLevel` – задаёт минимальный уровень логов
- `Override` – позволяет задать отдельные уровни для системных библиотек
- `Console` – вывод логов в терминал
- `File` – запись логов в файл
- `rollingInterval: Day` – создаётся новый файл каждый день
- `retainedFileCountLimit` – хранится только 5 последних файлов
- `fileSizeLimitBytes` – ограничение размера файла

5. Уровни логирования

В приложении используются основные уровни:

1. Debug

```
logger.LogDebug("Количество студентов: {studentsCount}", students.Count());
```

Рисунок 4. Уровень Debug

Используется для:

- отладки
- технической информации

2. Information

```
logger.LogInformation("Студенты найдены и отображены в списке.");
```

Рисунок 5. Уровень information

Используется для:

- обычных событий системы
- успешных операций

3. Warning

```
logger.LogWarning("Студент с id {id} не найден.", id);
```

Рисунок 6. Уровень Warning

4. Error

```
logger.LogError(ex, "Ошибка при создании студента");
```

Рисунок 7. Уровень Error

Используется для:

- исключений
- критических ошибок

6. Реализация логирования в CRUD операциях

GET – получение списка студентов:

```
logger.LogDebug("Количество студентов: {studentsCount}", students.Count());  
logger.LogInformation("Студенты найдены и отображены в списке.");
```

Рисунок 8. Реализация debug/information-логирования в GET

Если список пуст:

```
logger.LogWarning("Невозможно получить список студентов: список пуст.");
```

Рисунок 9. Реализация warning-логирования в GET

GET по ID:

```
logger.LogInformation("Студент найден | Id: {Id} | Имя: {Name}", student.Id, student.Name);
```

Рисунок 10. Реализация information-логирования в GET по ID

Если не найден:

```
logger.LogWarning("Студент с id {id} не найден.", id);
```

Рисунок 11. Реализация warning-логирования в GET по ID

Что фиксируется:

- факт поиска
- результат поиска

POST – создание студента

```
logger.LogInformation(  
    "Создание студента | Id: {Id} | Имя: {Name} | Время: {Time}",  
    student.Id,  
    student.Name,  
    DateTime.UtcNow);
```

Рисунок 12. Реализация information-логирования в POST

При ошибке:

```
logger.LogError(ex, "Ошибка при создании студента");
```

Рисунок 13. Реализация error-логирования в POST

Что фиксируется:

- создание сущности
- время операции
- исключения

PUT – обновление

```
logger.LogInformation("Данные студента с id {id} обновлены.", student.Id);
```

Рисунок 14. Реализация information-логирования в PUT

Если не найден:

```
logger.LogWarning("Студент с id {id} не найден.", id);
```

Рисунок 15. Реализация warning-логирования в PUT

DELETE – удаление

```
logger.LogInformation("Студент удалён | Id: {Id}", id);  
logger.LogDebug("Количество студентов: {studentsCount}", _students.Count());
```

Рисунок 16. Реализация information/debug-логирования в DELETE

Если не найден:

```
logger.LogWarning("Студент с id {id} не найден.", id);
```

Рисунок 17. Реализация warning-логирования в DELETE

Что фиксируется:

- удаление
- текущее состояние системы

7. Структурированное логирование

Используется формат:

```
logger.LogInformation("Студент найден | Id: {Id} | Имя: {Name}", student.Id, student.Name);
```

Рисунок 18. Структурированное логирование

Преимущества:

- Параметры сохраняются отдельно
- Удобно фильтровать логи
- Можно анализировать через системы мониторинга

8. Хранение и анализ логов

Логи сохраняются:

- в консоль — для разработки
- в файл logs/app-*.log — для анализа

Пример записи:

[2026-03-31 12:00:00 INF] StudentsController Студент найден | Id: 1 | Имя: Ivan

9. Скриншоты приложения

```
2026-03-31 16:08:28.988 +03:00 INF Microsoft.AspNetCore.Hosting.Diagnostics Request starting HTTP/1.1 POST http://localhost:5186/api/Students - application/json 100 ("EventId": ("Id": 1), "SpanId": "25f3672648eda97", "TraceId": "a72fddfab9a842c51809080fa027499", "ParentId": "0000000000000000", "ConnectionId": "0000000000000000", "RequestPath": "/api/Students")
2026-03-31 16:08:28.994 +03:00 INF Microsoft.AspNetCore.Routing.EndpointMiddleware Executing endpoint 'Lab10.Api.Students.Controllers.StudentsController.CreateStudent (Lab10)' ("EventId": ("Name": "ExecutingEndpoint"), "SpanId": "25f3672648eda97", "TraceId": "a72fddfab9a842c51809080fa027499", "ParentId": "0000000000000000", "ConnectionId": "0000000000000000", "RequestPath": "/api/Students")
2026-03-31 16:08:29.002 +03:00 INF Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker Route matched with {action = "CreateStudent", controller = "Students"}, Executing controller action with signature Microsoft.AspNetCore.Mvc.ActionResult CreateStudent(Lab10.Api.Students.Contracts.StudentRequestDTO) on controller Lab10.Api.Students.Controllers.StudentsController (Lab10) ("EventId": ("Id": 100, "Name": "ControllerActionExecuting"), "SpanId": "25f3672648eda97", "TraceId": "a72fddfab9a842c51809080fa027499", "ParentId": "0000000000000000", "ConnectionId": "0000000000000000", "RequestPath": "/api/Students", "ActionId": "6c39e3d6-275a-4b17-8034-dfccc0fcb94", "ActionName": "Lab10.Api.Students.Controllers.StudentsController.CreateStudent (Lab10)")
2026-03-31 16:08:29.028 +03:00 INF Lab10.Api.Students.Controllers.StudentsController Создание студента | Id: 1 | Имя: Hvahukhnpndzshyellssyaybzhmedtbp | Время: 03/31/2026 13:08:29 ("SpanId": "25f3672648eda97", "TraceId": "a72fddfab9a842c51809080fa027499", "ParentId": "0000000000000000", "ConnectionId": "0000000000000000", "RequestPath": "/api/Students", "ActionId": "6c39e3d6-275a-4b17-8034-dfccc0fcb94", "ActionName": "Lab10.Api.Students.Controllers.StudentsController.CreateStudent (Lab10)")
2026-03-31 16:08:29.034 +03:00 INF Microsoft.AspNetCore.Mvc.Infrastructure.ObjectResultExecutor Executing CreatedActionResult, writing value of type 'Lab10.Api.Students.Contracts.StudentResponseDTO' ("EventId": ("Id": 1, "Name": "ObjectResultExecuting"), "SpanId": "25f3672648eda97", "TraceId": "a72fddfab9a842c51809080fa027499", "ParentId": "0000000000000000", "ConnectionId": "0000000000000000", "RequestPath": "/api/Students", "ActionId": "6c39e3d6-275a-4b17-8034-dfccc0fcb94", "ActionName": "Lab10.Api.Students.Controllers.StudentsController.CreateStudent (Lab10)")
2026-03-31 16:08:29.051 +03:00 INF Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker Executed action Lab10.Api.Students.Controllers.StudentsController.CreateStudent (Lab10) in 82.972ms ("EventId": ("Id": 100, "Name": "ActionExecuted"), "SpanId": "25f3672648eda97", "TraceId": "a72fddfab9a842c51809080fa027499", "ParentId": "0000000000000000", "ConnectionId": "0000000000000000", "RequestPath": "/api/Students")
2026-03-31 16:08:29.059 +03:00 INF Microsoft.AspNetCore.Routing.EndpointMiddleware Executed endpoint 'Lab10.Api.Students.Controllers.StudentsController.CreateStudent (Lab10)' ("EventId": ("Id": 1, "Name": "ExecutedEndpoint"), "SpanId": "25f3672648eda97", "TraceId": "a72fddfab9a842c51809080fa027499", "ParentId": "0000000000000000", "ConnectionId": "0000000000000000", "RequestPath": "/api/Students")
2026-03-31 16:08:29.063 +03:00 INF Microsoft.AspNetCore.Hosting.Diagnostics Request finished HTTP/1.1 POST http://localhost:5186/api/Students - 200 null application/json; charset=utf-8 79.501ms ("EventId": ("Id": 1), "SpanId": "25f3672648eda97", "TraceId": "a72fddfab9a842c51809080fa027499", "ParentId": "0000000000000000", "ConnectionId": "0000000000000000", "RequestPath": "/api/Students")
```

Рисунок 19. Логирование POST в консоли

Вывод

В ходе выполнения работы:

- изучена архитектура логирования в ASP.NET Core
- освоена библиотека Serilog
- реализовано логирование всех CRUD операций
- настроено хранение логов в файлах и консоли
- применено структурированное логирование

Итог: логирование является критически важной частью backend-приложений, обеспечивая контроль, диагностику и устойчивость системы.

Листинг кода

Листинг 1. Файл Program.cs:

```
using Serilog;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddSwaggerGen();

builder.Services.AddControllers();

builder.Services.AddLogging(l =>
{
    l.ClearProviders(); // удаление стандартных провайдеров
    l.AddSerilog(new LoggerConfiguration()
        .ReadFrom.Configuration(builder.Configuration)
        .CreateLogger());
});

var app = builder.Build();

app.UseSwagger().UseSwaggerUI();

app.MapControllers();

app.Run();
```

Листинг 2. Файл StudentRequestDTO.cs:

```
using System.ComponentModel.DataAnnotations;

namespace Lab9.Api.Students.Contracts
{
    public class StudentRequestDTO
    {
        [Required]
        [MaxLength(40)]
        [MinLength(2)]
        [RegularExpression(@"^([A-ZА-Я][a-za-я]*[ ]*)+$")]
        public required string Name { get; init; }
        [Required]
        [Range(1, 100)]
        public required int Group { get; init; }
        [Required]
        public required string Specialization { get; init; }
    }
}
```

Листинг 3. Файл StudentResponseDTO.cs:

```
namespace Lab9.Api.Students.Contracts
{
    public class StudentResponseDTO
    {
        public required int Id { get; init; }
        public required string Name { get; init; }
        public required string Specialization { get; init; }
        public required int Group { get; init; }
    }
}
```

```
}
```

Листинг 4. Файл StudentsController.cs:

```
using Lab10.Api.Students.Contracts;
using Lab10.Api.Students.Models;
using Microsoft.AspNetCore.Mvc;

namespace Lab10.Api.Students.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class StudentsController(ILogger<StudentsController> logger) :
        ControllerBase
    {
        // GET метод
        [HttpGet]
        public ActionResult<StudentResponseDTO[]> GetAllStudents()
        {
            var students = _students.Select(x => new StudentResponseDTO
            {
                Id = x.Id,
                Name = x.Name,
                Group = x.Group,
                Specialization = x.Specialization
            }).ToArray();

            if (students.Any())
            {
                logger.LogDebug("Количество студентов: {studentsCount}",
students.Count());
                logger.LogInformation("Студенты найдены и отображены в списке.");
                return Ok(students);
            }

            logger.LogWarning("Невозможно получить список студентов: список пуст.");
            return NotFound("Список студентов пуст.");
        }

        // GET{id} метод
        [HttpGet]
        [Route("{id}")]
        public ActionResult<StudentResponseDTO> GetStudent(int id)
        {
            var student = _students.FirstOrDefault(x => x.Id == id);
            if (student == null)
            {
                logger.LogWarning("Студент с id {id} не найден.", id);
                return NotFound("Студент не найден.");
            }

            logger.LogInformation("Студент найден | Id: {Id} | Имя: {Name}",
student.Id, student.Name);
            return Ok(new StudentResponseDTO
            {
                Id = student.Id,
                Name = student.Name,
                Group = student.Group,
                Specialization = student.Specialization
            });
        }

        // POST метод
        [HttpPost]
```

```

public ActionResult CreateStudent([FromBody] StudentRequestDTO contract)
{
    try
    {
        var id = _students.Count == 0 ? 1 : _students.Max(x => x.Id) + 1;

        var student = new Student
        {
            Id = id,
            Name = contract.Name,
            Group = contract.Group,
            Specialization = contract.Specialization,
        };

        logger.LogInformation(
            "Создание студента | Id: {Id} | Имя: {Name} | Время: {Time}",
            student.Id,
            student.Name,
            DateTime.UtcNow);
        _students.Add(student);

        return CreatedAtAction(nameof(GetStudent), new { id = student.Id },
new StudentResponseDTO
        {
            Id = student.Id,
            Name = student.Name,
            Group = student.Group,
            Specialization = student.Specialization
        });
    }
    catch (Exception ex)
    {
        logger.LogError(ex, "Ошибка при создании студента");
        return StatusCode(500);
    }
}

// PUT{id} метод
[HttpPut("{id}")]
public ActionResult UpdateStudent(int id, [FromBody] StudentRequestDTO
contract)
{
    var student = _students.FirstOrDefault(x => x.Id == id);

    if (student == null)
    {
        logger.LogWarning("Студент с id {id} не найден.", id);
        return NotFound();
    }

    student.Name = contract.Name;
    student.Group = contract.Group;
    student.Specialization = contract.Specialization;

    logger.LogInformation("Данные студента с id {id} обновлены.", student.Id);
    return NoContent();
}

// DELETE{id} метод
[HttpDelete]
[Route("{id}")]
public ActionResult DeleteStudent(int id)
{
    var index = _students.FindIndex(x => x.Id == id);
    if (index < 0)

```

```

        {
            logger.LogWarning("Студент с id {id} не найден.", id);
            return NotFound("Студент не найден.");
        }

        _students.RemoveAt(index);
        logger.LogInformation("Студент удалён | Id: {Id}", id);
        logger.LogDebug("Количество студентов: {studentsCount}",
            _students.Count());
        return NoContent();
    }

    private static List<Student> _students = new();
}

```

Листинг 5. Файл StudentModel.cs:

```

namespace Lab9.Api.Students.Models
{
    public class Student
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public int Group { get; set; }
        public string Specialization { get; set; }
    }
}

```

Листинг 6. Файл appsettings.json:

```

{
  "Serilog": {
    "MinimumLevel": {
      "Default": "Debug",
      "Override": {
        "Microsoft": "Debug",
        "Microsoft.AspNetCore": "Information",
        "System": "Debug"
      }
    },
    "WriteTo": [
      {
        "Name": "Console",
        "Args": {
          "outputTemplate": "[{Timestamp:yyyy-MM-dd HH:mm:ss.fff zzz}
{Level:u3}] {SourceContext} {Message:l} {Properties:j}{NewLine}{Exception}"
        }
      },
      {
        "Name": "File",
        "Args": {
          "path": "logs/app-.log",
          "rollingInterval": "Day",
          "retainedFileCountLimit": 5,
          "rollOnFileSizeLimit": true,
          "fileSizeLimitBytes": 104857600,
          "outputTemplate": "[{Timestamp:yyyy-MM-dd HH:mm:ss.fff zzz}
{Level:u3}] {SourceContext} {Message:l} {Properties:j}{NewLine}{Exception}"
        }
      }
    ]
  },
}

```

```
}    "AllowedHosts": "*"
}
```